

Object oriented programming

Object-Oriented Programming (OOP) in Python is a programming paradigm where code is organized around objects, which are instances of classes. Each object encapsulates data and behavior, and classes serve as blueprints for creating objects with predefined attributes and methods. OOP promotes modularity, code reuse, and easy maintenance by modeling real-world entities as objects with well-defined properties and interactions.

**** paradigm** - refers to a fundamental style or approach to writing code.

**** "Encapsulates"** - refers to the concept of bundling data (attributes) and methods (functions) that operate on that data within a single unit, typically a class in the context of Object-Oriented Programming (OOP).

lets discuss in terms of a car

CLASS:

In Object-Oriented Programming (OOP), you can represent real-world entities, class-such as a car, as objects. This means you can create a blueprint for what a car is and what it can do within your program.

Objects:

In the context of OOP, an object is a concrete instance of a class. For our example, a car object would represent a specific car, like a Toyota Camry or a Ford Mustang.

Attributes (variable/parameter):

Objects have attributes, which represent the characteristics or properties of the object. For a car object, attributes could include things like the make (Toyota), model (Camry), color (red), and current speed (60 mph).

Methods (Behavior):

Objects also have methods, which represent the actions or behaviors that the object can perform. For a car object, methods could include actions like starting the engine, accelerating, braking, and turning.

what if class is DOG

CLASS:

In this case, the real-world entity is a dog. We want to represent dogs in our program as objects.

Object:

Each object represents a specific dog. For example, you might have objects representing different breeds or individual dogs with unique characteristics.

Attributes (VARIABLES):

Attributes of a dog object could include characteristics such as breed, color, age, weight, and name.

Methods (Behavior):

Methods of a dog object could include behaviors such as barking, eating, walking, running, and sleeping.

LETS DISCUSS IN GENERAL

Class:

A class is a blueprint or template for creating objects. It defines the attributes and methods that objects of that class will have. Classes serve as a blueprint for creating multiple instances (objects) with similar characteristics and behaviors. Object:

object

is a concrete instance of a class. It represents a specific entity or concept in the real world. Objects have attributes (data) and methods (behavior) associated with them, as defined by their class. Each object is independent and encapsulates its own state and behavior. Attributes:

Attributes / Variables

are the properties or characteristics associated with an object. They represent the state of the object and describe its properties. Attributes are defined within a class and are accessed using dot notation. Examples of attributes for a "Car" class could include make, model, year, color, etc. Methods:

Methods

are functions that are defined within a class and are associated with objects of that class. They represent the behavior or actions that objects of the class can perform. Methods operate on the

attributes of the object and can modify its state. Methods are invoked using dot notation and typically take the object itself (referred to as `self` in Python) as the first parameter.

basic example-1

```
In [10]: class student: # class ----dog
        # Define class-level attributes
        name = None #-----breed
        roll_number = None #-----color
        #we define two class-level attributes (name and roll_number). These attributes are assigned to the class
        student1 = Student() #object-1
        student1.name = "Alice" # object level attribute-1
        student1.roll_number = 101 # object level attribute-2

        student2 = Student() #object 2
        student2.name = "Bob" # object level attribute
        student2.roll_number = 102 # object level attribute-

        student3 = Student() #object 3
        student3.name = "Charlie" # object level attribute-
        student3.roll_number = 103 # object level attribute-

        # Accessing attributes directly
        print("Name:", student1.name, "Roll Number:", student1.roll_number)
        print("Name:", student2.name, "Roll Number:", student2.roll_number)
        print("Name:", student3.name, "Roll Number:", student3.roll_number)
```

```
Name: Alice Roll Number: 101
Name: Bob Roll Number: 102
Name: Charlie Roll Number: 103
```

example 2

init Method:

The **init** method is defined within a class to initialize the attributes of objects when they are created.

self Parameter:

`self` is a reference to the (instance of the class)/ OBJECT It is the first parameter of any method defined within a class (including **init**).

When you call a method on an object, Python automatically passes the object itself as the first argument to the method.

```
In [18]: class TenAStudent:
        def __init__(self, roll_number, name, sex): #3 ATTRIBUTES- properties or characteristics
            self.roll_number = roll_number
            self.name = name
            self.sex = sex
```

```

def display_info(self): #OBJECTS
    print("Class: 10 A")
    print("Roll Number:", self.roll_number) #Instance ATTRIBUTES
                                         #"Instance attributes" specifically re

    print("Name:", self.name)
    print("Sex:", self.sex)

# Creating instances of the TenAStudent class (objects) and setting attributes
student1 = TenAStudent(101, "Alice", "Female")
student2 = TenAStudent(102, "Bob", "Male")
student3 = TenAStudent(103, "Charlie", "Male")
# Accessing attributes and calling methods
print("Student 1 Information:")
student1.display_info()
print("\nStudent 2 Information:")
student2.display_info()
print("\nStudent 3 Information:")
student3.display_info()

display_info() # to get the complete attributes

```

Student 1 Information:
Class: 10 A
Roll Number: 101
Name: Alice
Sex: Female

Student 2 Information:
Class: 10 A
Roll Number: 102
Name: Bob
Sex: Male

Student 3 Information:
Class: 10 A
Roll Number: 103
Name: Charlie
Sex: Male

```

-----
NameError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_21520\2170554207.py in <module>
    24 student3.display_info()
    25
--> 26 display_info() # to get the complete attributes

NameError: name 'display_info' is not defined

```

In [3]: *#NB: self is just a representation, we can use another variable btw.*

```

In [19]: class TenAStudent:
    def __init__(X, roll_number, name, sex): #3 ATTRIBUTES- properties or characteristics
        X.roll_number = roll_number
        X.name = name
        X.sex = sex

    def display_info(X): #OBJECTS
        print("Class:", "10A")
        print("Roll Number:", X.roll_number) #Instance ATTRIBUTES
                                         #"Instance attributes" specifically re

```

```

        print("Name:", X.name)
        print("Sex:", X.sex)

# Creating instances of the TenAStudent class (objects) and setting attributes
student1 = TenAStudent(101, "Alice", "Female")
student2 = TenAStudent(102, "Bob", "Male")
student3 = TenAStudent(103, "Charlie", "Male")

# Accessing attributes and calling methods
print("Student 1 Information:")
student1.display_info()
print("\nStudent 2 Information:")
student2.display_info()
print("\nStudent 3 Information:")
student3.display_info()

```

```

Student 1 Information:
Class: 10A
Roll Number: 101
Name: Alice
Sex: Female

```

```

Student 2 Information:
Class: 10A
Roll Number: 102
Name: Bob
Sex: Male

```

```

Student 3 Information:
Class: 10A
Roll Number: 103
Name: Charlie
Sex: Male

```

Inheritance

Inheritance is a fundamental concept in object-oriented programming (OOP) where a class (known as the child class or subclass) can inherit attributes and methods from another class (known as the parent class or superclass). This allows the child class to reuse and extend the functionality defined in the parent class.

Here's a breakdown of key points about inheritance:

Parent Class (Superclass):

The parent class is a class that defines attributes and methods that can be inherited by other classes.

The parent class can also be referred to as the base class or superclass.

Child Class (Subclass):

The child class is a class that inherits attributes and methods from a parent class. It can add new attributes or methods, override existing methods, or inherit methods without modification. The child class extends or specializes the behavior of the parent class.

Inheritance Syntax:

In Python, inheritance is implemented using the syntax class

ChildClassName(ParentClassName):.

The child class is defined with parentheses containing the name of the parent class from which it inherits.

```
In [21]: class Animal: #parent
          def speak(self):
              print("Animal speaks")

          class Dog(Animal): #child
              def bark(self):
                  print("Dog CRY")

          # Creating an instance of Dog class
          dog = Dog()#child
          dog.speak() # Output: "Animal speaks"
          dog.bark() # Output: "Dog barks"
```

Animal speaks
Dog CRY

```
In [20]: class Animal: #parent
          def speak(self):
              print("Animal speaks")

          class Dog(Animal): #child
              def bark(self):
                  print("Dog barks")

          # Creating an instance of Dog class
          #child
          Dog().speak() # Output: "Animal speaks"
          Dog().bark() # Output: "Dog barks"
```

Animal speaks
Dog barks

Single Inheritance

Multilevel Inheritance

Multiple Inheritance

Hybrid Inheritance

Hierarchical Inheritance

multilevel

```
In [13]: class Animal: #parent
          def speak(self):
              print("Animal speaks")

          # Dog inherits from Animal
          class Dog(Animal):
              def bark(self):
                  print("Dog barks")

          # Bulldog inherits from Dog
          class Bulldog(Dog):
              def guard(self):
                  print("Bulldog guards")

          # Creating an instance of Bulldog class
          bulldog = Bulldog()
          bulldog.speak() # Output: "Animal speaks" (Inherited from Animal)
          bulldog.bark() # Output: "Dog barks" (Inherited from Dog)
          bulldog.guard() # Output: "Bulldog guards" (Defined in Bulldog)
```

Animal speaks

Dog barks

Bulldog guards

Polymorphism:

Concept: Polymorphism refers to the ability of objects of different classes to respond to the same message or method invocation in different ways. It allows objects to be treated uniformly based on their common interface, while providing flexibility in behavior based on their specific implementations.

Encapsulation:

Concept: Encapsulation is the bundling of data (attributes) and methods that operate on the data into a single unit or class. It hides the internal state of an object from the outside world and restricts direct access to the object's components. Encapsulation promotes data security, modular design, and code maintainability by enforcing well-defined interfaces for interacting with objects.

In []:

REFERENCE

In [22]:

```
# List of student instances
students = [student1, student2, student3]

# Function to find student by roll number
def find_student_by_roll_number(roll_number):
    for student in students:
        if student.roll_number == roll_number:
            return student
    return None

# Call the function to find the student with roll number 101
student_101 = find_student_by_roll_number(101)

# If the student with roll number 101 exists, print their name
if student_101:
    print("Student with roll number 101:", student_101.name)
else:
    print("Student with roll number 101 not found")
```

Student with roll number 101: Alice

In []: